

Joint Optimization of Energy, Latency and Age of Information for MEC Task Offloading Using TLBO-HHO Algorithm

1 st Given Name Surname	2 nd Given Name Surname	3 rd Given Name Surname
dept. name of organization (of Aff.)	dept. name of organization (of Aff.)	dept. name of organization (of Aff.)
name of organization (of Aff.)	name of organization (of Aff.)	name of organization (of Aff.)
City, Country	City, Country	City, Country
email address or ORCID	email address or ORCID	email address or ORCID

摘要—移动边缘计算 (MEC) 通过将计算资源部署在网络边缘, 为资源受限的移动设备提供低延迟、高能效的计算服务。然而, 在多用户场景下, 如何有效进行任务卸载决策并同时优化系统能耗、时延和信息新鲜度仍是一个具有挑战性的问题。本文提出了一种融合教学优化 (TLBO) 与哈里斯鹰优化 (HHO) 的混合元启发式算法 (TLBO-HHO), 用于解决 MEC 环境中考虑信息年龄 (Age of Information, AoI) 的任务卸载与资源分配优化问题。主要贡献包括: (1) 建立了综合考虑能耗、时延和 AoI 的三目标优化模型, 采用排队论分析任务等待时间, 并证明其为 NP-hard 问题; (2) 设计了基于双阶段集成的 TLBO-HHO 融合算法, 通过自适应切换机制平衡全局探索与局部开发能力; (3) 实验结果表明, 相比原始 TLBO、GA 和 GWO 算法, TLBO-HHO 在适应度值上分别提升了 65.2%、85.4% 和 74.6%, 同时有效降低了信息过时性。

Index Terms—移动边缘计算; 任务卸载; 资源分配; 信息年龄; 教学优化算法; 哈里斯鹰优化; 双阶段集成。

I. 引言

随着物联网 (IoT) 和 5G/6G 通信技术的快速发展, 移动设备产生的数据量呈指数级增长。移动边缘计算 (MEC) 通过将计算资源部署在网络边缘, 为延迟敏感型应用提供了有效解决方案 [1]。在 MEC 系统中, 任务卸载决策直接影响系统性能, 需要在能耗和延迟之间进行权衡。近年来, 随着实时通信和实时决策需求的提升, Age of Information (AoI) 已成为衡量系统信息新鲜度的重要指标 [15], 加入 AoI 优化成为了研究趋势。

现有研究方法包括凸优化 [2]、博弈论、深度强化学习 [3] 和元启发式算法等。然而, 单一元启发式算法存在固有缺陷: 遗传算法 (GA) 易早熟收敛, 灰狼优化 (GWO) 收敛速度慢 [4]。混合元启发式算法通过结合不同算法的优势能够有效克服这些局限性。

本文的主要贡献如下:

- 建立了综合考虑能耗、延迟和 AoI 的三目标优化模型, 采用 0.25-0.35-0.40 权重设置, 通过排队论分析任务等待时间, 并证明其 NP-hard 特性
- 提出了基于双阶段集成的 TLBO-HHO 融合算法, 通过自适应概率机制 (0.7→0.1) 实现探索与开发的动态平衡
- 在包含 20 设备、4 边缘服务器、2 云服务器和 40 任务的场景中, 验证了算法在信息时效性优化方面的有效性

II. 相关工作

A. 元启发式优化算法

Rao [5] 提出的 TLBO 算法模拟教学过程, 具有参数少、收敛稳定的优点。Heidari 等 [6] 提出的 HHO 算法通过模拟哈里斯鹰狩猎行为, 在处理多峰函数时成功率达 85-95%。最新研究中, Chen 等 [7] 和 Tu 等 [8] 分别提出了 HHO 的混合改进版本, 但 TLBO-HHO 在 MEC 中的应用仍然稀缺。

B. MEC 任务卸载优化

任务卸载优化方法可分为三大类：

(1) 数学优化方法：Wang 等 [9] 提出了基于拉格朗日对偶分解的联合计算卸载和干扰管理方案，在理论上保证了了解的最优性。Liu 等 [10] 设计了基于 Lyapunov 优化的延迟最优任务调度算法，能够在保证队列稳定性的同时最小化平均延迟。这类方法理论基础扎实但难以处理非凸问题。

(2) 机器学习方法：Wang 等 [11] 提出了基于深度强化学习的动态轨迹控制方案，实现了 UAV 辅助 MEC 中的自适应资源分配。Zang 等 [13] 设计了联邦深度强化学习框架，在保护用户隐私的同时优化任务卸载决策。这类方法适应性强但需要大量训练数据。

(3) 元启发式方法：Li 和 Zhu [12] 应用遗传算法优化 MEC 中的任务卸载，实现了能耗降低 30%。Ali 等 [14] 提出了多目标 HHO 算法用于云雾计算中的任务调度，在处理 NP-hard 问题时表现良好，能够实现高达 56% 的能耗降低。

C. 信息年龄 (AoI) 优化

AoI 作为衡量信息新鲜度的关键指标，最早由 Kaul 等 [16] 在 2012 年提出，用于量化接收端获得的信息的时效性。在 IoT 和 MEC 场景中，AoI 的重要性日益凸显。Abd-Elmagid 等 [15] 全面综述了 AoI 在物联网中的作用，指出在状态更新系统中优化 AoI 对于实时决策至关重要。

Liu 等 [17] 研究了车载边缘计算中的 AoI 感知任务计算问题，提出了异步深度强化学习方法，能够同时优化计算资源分配和 AoI 性能。Zhou 等 [18] 设计了考虑 AoI 的契约匹配机制，在车载雾计算环境中实现了计算资源的高效分配。然而，现有研究很少将 AoI 与元启发式算法结合用于 MEC 任务卸载优化，这正是本文的创新点之一。

III. 系统模型与问题陈述

A. 网络架构与场景描述

考虑一个典型的三层 MEC 网络架构，如图 1 所示。该架构反映了实际部署中的层次化计算资源分布，与商业 MEC 平台（如 AWS Wavelength、Azure Edge Zones）的部署模式一致。

系统包含以下组件：

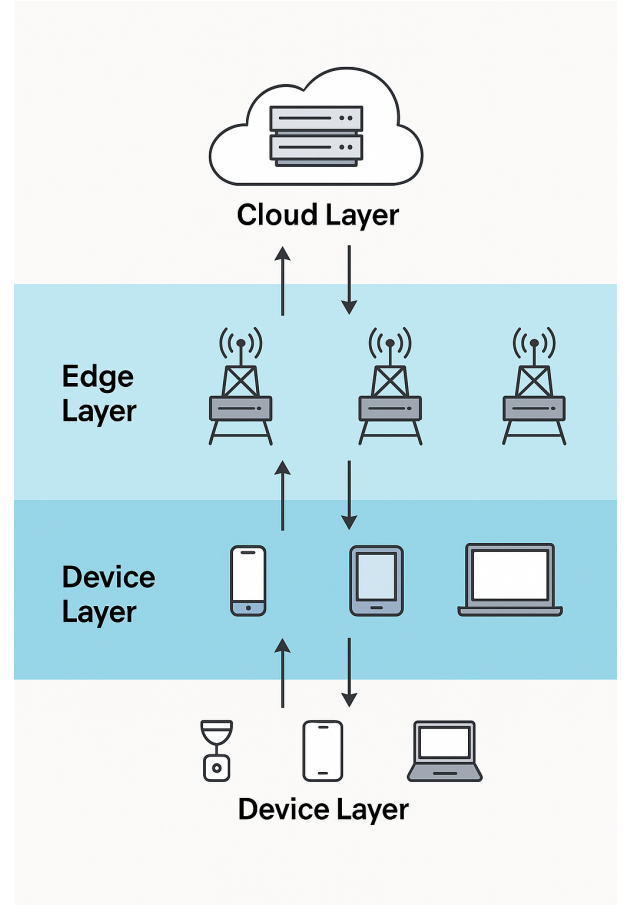


图 1. 设备层、边缘层、云层网络架构。

- 设备层： $\mathcal{D} = \{1, 2, \dots, D\}$ 表示 $D = 20$ 个移动设备，包括低端 IoT 设备 (40%, 0.2-0.5 GHz)、中端移动设备 (40%, 0.8-1.5 GHz) 和高端设备 (20%, 2.0-3.0 GHz)
- 边缘层： $\mathcal{E} = \{1, 2, \dots, E\}$ 表示 $E = 4$ 个边缘服务器 (4.0-6.0 GHz)
- 云层： $\mathcal{C} = \{1, 2, \dots, C\}$ 表示 $C = 2$ 个云服务器 (8.0-12.0 GHz)
- 任务集： $\mathcal{T} = \{1, 2, \dots, T\}$ 表示 $T = 40$ 个待处理任务，包括计算密集型 (30%)、数据密集型 (25%)、实时敏感型 (25%) 和轻量级 (20%)

B. 计算模型

1) 任务特征：每个任务 $i \in \mathcal{T}$ 由五元组表征：

$$\tau_i = (D_i, C_i, T_i^{\max}, \lambda_i, AoI_i^{\max}) \quad (1)$$

其中：

- D_i 为输入数据量 (MB)， D_i^{result} 为输出数据量

- C_i 为计算复杂度 (CPU cycles/bit)
- $T_i^{\max}$ 为最大可容忍延迟 (s)
- λ_i 为任务更新间隔 (s)
- $AoI_i^{\max}$ 为最大可接受信息年龄 (s)

2) 延迟模型: 任务处理总延迟包括传输延迟、等待延迟和计算延迟:

$$T_i = T_i^{\text{trans}} + T_i^{\text{wait}} + T_i^{\text{comp}} \quad (2)$$

本地执行延迟仅包含计算时间:

$$T_{i,d(i)}^{\text{local}} = \frac{D_i \cdot C_i}{f_{i,d(i)}} \quad (3)$$

其中, $f_{i,d(i)}$ 是分配给任务 i 的 CPU 频率。

任务卸载到边缘服务器的延迟:

$$T_{i,d(i),e(i)}^{\text{edge}} = \frac{D_i}{R_{d(i),e(i)}} + T_{\text{queue}}^{\text{edge}} + \frac{D_i \cdot C_i}{f_{i,e(i)}} + \frac{D_i^{\text{result}}}{R_{e(i),d(i)}} \quad (4)$$

其中, $R_{j,k}$ 是节点 j 到 k 的传输速率, 采用香农公式计算:

$$R_{j,k} = B_{j,k} \log_2 \left(1 + \frac{P_j^{\text{trans}} \cdot h_{j,k}}{N_0 \cdot B_{j,k}} \right) \quad (5)$$

排队延迟分析: 考虑边缘服务器采用 M/M/1 排队模型, 平均等待时间为:

$$T_{\text{queue}}^{\text{edge}} = \frac{\rho}{f_e \cdot (1 - \rho)} \quad (6)$$

其中, $\rho = \lambda_e / \mu_e$ 是服务器利用率, λ_e 是任务到达率, μ_e 是服务率。

3) 能耗模型: 计算能耗采用动态功耗模型:

$$E_{i,j}^{\text{comp}} = \kappa_j \cdot (f_{i,j})^\alpha \cdot D_i \cdot C_i \quad (7)$$

其中, $\kappa_j \in [1.0, 3.5] \times 10^{-27}$, $\alpha \in [2.2, 2.8]$ 根据设备类型设定。

传输能耗: $E_{i,j,k}^{\text{trans}} = P_j^{\text{trans}} \cdot (D_i + D_i^{\text{result}}) / R_{j,k}$, 其中 $P_j^{\text{trans}} \in [0.1, 0.5] \text{W}$ 。

移动设备的总能耗:

$$E_i = x_i^{\text{local}} \cdot E_{i,d(i)}^{\text{comp}} + x_i^{\text{edge}} \cdot E_{i,d(i),e(i)}^{\text{trans}} + x_i^{\text{cloud}} \cdot (E_{i,d(i),e(i)}^{\text{trans}} + E_{i,e(i),c(i)}^{\text{trans}}) \quad (8)$$

4) AoI 模型: 为了描述信息的新鲜程度, 我们定义了 Age of Information (AoI) 指标。对于任务 i , 其在时间点 t 的 AoI 定义为:

$$AoI_i(t) = t - U_i(t) \quad (9)$$

其中, $U_i(t)$ 为任务 i 最后一次成功完成更新的时刻。

考虑任务处理延迟和更新间隔, 任务 i 的平均 AoI 为:

$$\overline{AoI}_i = \lambda_i + \frac{T_i}{2} + \frac{T_i^2}{2\lambda_i} \quad (10)$$

该公式考虑了三个部分: 更新间隔 λ_i 、平均等待时间 $T_i/2$, 以及由于处理延迟导致的额外老化 $T_i^2/(2\lambda_i)$ 。

C. 优化问题建模

1) 决策变量: 定义二进制决策变量 $\mathbf{x} = \{x_i^{\text{local}}, x_i^{\text{edge}}, x_i^{\text{cloud}}\}_{i \in \mathcal{T}}$, 其中:

- x_i^{local} 表示任务 i 在本地设备执行
- x_i^{edge} 表示任务 i 卸载到边缘服务器
- x_i^{cloud} 表示任务 i 卸载到云服务器

连续决策变量 $\mathbf{f} = \{f_{i,j}\}_{i \in \mathcal{T}, j \in \mathcal{D} \cup \mathcal{E} \cup \mathcal{C}}$ 表示分配的 CPU 频率。

2) 目标函数: 基于实际应用需求和最新研究趋势, 采用加权和方法将三目标转化为单目标:

$$\min_{\mathbf{x}, \mathbf{f}} F = \sum_{i \in \mathcal{T}} (w^E \cdot \frac{E_i}{E^{\text{norm}}} + w^T \cdot \frac{T_i}{T^{\text{norm}}} + w^{AoI} \cdot \frac{\overline{AoI}_i}{AoI^{\text{norm}}}) \quad (11)$$

其中:

- 权重设置: $w^E = 0.25$, $w^T = 0.35$, $w^{AoI} = 0.40$ (信息时效性优先)
- 归一化因子: $E^{\text{norm}} = \max_{i \in \mathcal{T}} E_i^{\text{local}}$, $T^{\text{norm}} = \max_{i \in \mathcal{T}} T_i^{\text{cloud}}$, $AoI^{\text{norm}} = \max_{i \in \mathcal{T}} AoI_i^{\text{max}}$

3) 约束条件: 主要约束包括: (1) 任务分配唯一性: $x_i^{\text{local}} + x_i^{\text{edge}} + x_i^{\text{cloud}} = 1, \forall i$; (2) 资源容量: $\sum_i x_i^j \cdot f_{i,j} \leq F_j^{\text{max}}$; (3) 延迟约束: $T_i \leq T_i^{\text{max}}$; (4) AoI 约束: $\overline{AoI}_i \leq AoI_i^{\text{max}}$; (5) QoS 约束: 95% 任务满足延迟要求。

4) 问题复杂度分析: **定理 1:** 考虑 AoI 的 MEC 任务卸载与资源分配问题是 NP-hard。

证明: 考虑一个特殊实例: 仅有一个边缘服务器 ($E = 1$), 所有任务具有相同的数据量 D 和计算复杂度 C , 且忽略传输延迟和 AoI 约束。此时, 问题等价于

$$\min_{S \subseteq \mathcal{T}} \sum_{i \in S} p_i, \quad \text{s.t.} \quad p_i = \frac{DC}{f_i}, \forall i \in S, \quad \sum_{i \in S} f_i \leq F^{\text{max}}, \quad f_i \geq 0, \forall i \in S \quad (12)$$

其中 S 为被选中在边缘执行的任务集合, $F^{\max}$ 为服务器可用 CPU 周期总量。

定义每个任务 i 的”重量” $w_i = f_i$, ”价值” v_i 为

$$v_i = \frac{1}{p_i} = \frac{f_i}{DC} \quad (13)$$

则式 (24) 转化为在容量 $F^{\max}$ 的背包中选择物品以最大化总价值的经典 Knapsack 问题, 而 Knapsack 已被证明为 NP-hard。

此外, 完整模型中既含二进制变量 $\mathbf{x}$, 又含连续变量 $\mathbf{f}$, 且目标函数包含非线性项 (如 f_i^α), 同时需要考虑 AoI 约束的时序依赖性, 因此属于混合整数非线性规划 (MINLP), 求解难度进一步加大。

IV. 融合 TLBO-HHO 算法设计

A. 算法动机与创新设计理念

TLBO 通过教师-学生交互机制实现稳定的群体进化, HHO 通过模拟鹰群狩猎行为实现动态搜索。本文提出的 TLBO-HHO 融合算法采用双阶段集成架构: (1) 双阶段协同机制, TLBO 引导全局探索, HHO 提供局部开发; (2) 自适应参数协调; (3) AoI 感知的约束处理。

B. 解的编码与初始化

每个解 S_k 编码为混合向量: $S_k = [\mathbf{x}_k, \mathbf{f}_k, \mathbf{a}_k]$, 其中 $x_i^k \in \{0, 1, 2\}$ 表示任务执行位置, f_i^k 为 CPU 频率, a_i^k 为服务器分配。采用 30% 启发式和 70% 随机的智能初始化策略。

Algorithm 1 智能初始化策略

Require: 种群规模 P

Ensure: 初始化后的种群 $\mathcal{P}$

```

1:  $\mathcal{P} \leftarrow \emptyset$ 
2:  $n_{heur} \leftarrow \lfloor 0.3 \times P \rfloor$  // 启发式部分
3: for  $i = 1$  to  $n_{heur}$  do
4:    $s \leftarrow \text{HEURISTICINIT}()$ 
5:    $\mathcal{P} \leftarrow \mathcal{P} \cup \{s\}$ 
6: end for
7:  $n_{rand} \leftarrow P - n_{heur}$  // 随机部分
8: for  $i = 1$  to  $n_{rand}$  do
9:    $s \leftarrow \text{RANDOMINIT}()$ 
10:   $\mathcal{P} \leftarrow \mathcal{P} \cup \{s\}$ 
11: end for
12: return  $\mathcal{P}$ 

```

C. 改进的 TLBO 和 HHO 机制

TLBO 采用自适应教学因子: $T_F = 1 + \text{rand}() \cdot (1 + e^{-\sigma_{pop}/\sigma_{max}})$, 增强的教师阶段考虑精英个体影响。HHO 引入非线性逃逸能量: $E = 2E_0 \cdot \cos(\pi t/2T_{max}) \cdot (1 + 0.1 \sin(4\pi t/T_{max}))$, 避免早熟收敛 (见算法 2)。

D. 双阶段集成与 AoI 优化

自适应概率切换: $p_{HHO} = p_{base} \cdot \alpha_{conv} \cdot \beta_{div}$, 其中 $p_{base} = 0.7 - 0.6(t/T_{max})$ 。AoI 感知的适应度函数:

$$\text{Fitness}(X) = w^E \cdot E(X) + w^T \cdot D(X) + w^{AoI} \cdot \text{AoI}(X) + \text{Penalty}(X) \quad (14)$$

Algorithm 2 TLBO-HHO 双阶段集成算法 (含 AoI 优化)

Require: 种群大小 N , 最大迭代数 T_{max} , 权重 w^E, w^T, w^{AoI}

Ensure: 最优解 $\mathbf{X}_{best}$

```

1:  $P \leftarrow \text{INTELLIGENTINIT}(N)$ ;  $\text{ElitePool} \leftarrow \emptyset$ 
2: 评估所有个体适应度 (含 AoI)
3: for  $t = 1$  to  $T_{max}$  do
4:    $p_{HHO} \leftarrow \text{ADAPTIVEPROB}(t, P)$ 
5:    $E \leftarrow \text{ESCAPEENERGY}(t)$ ;  $T_F \leftarrow \text{TEACHINGFACTOR}(P)$ 
6:   for each  $\mathbf{X}_i \in P$  do
7:     if  $\text{rand}() < p_{HHO}$  then
8:       if  $|E| \geq 1$  then
9:          $\mathbf{X}_{new} \leftarrow \text{HHO\_EXPLORE}(\mathbf{X}_i, \mathbf{X}_{best})$ 
10:      else
11:         $\mathbf{X}_{new} \leftarrow \text{HHO\_EXPLOIT}(\mathbf{X}_i, \mathbf{X}_{best}, E, J)$ 
12:      end if
13:    else
14:       $\mathbf{X}_{teacher} \leftarrow \text{SELECTTEACHER}(P, \text{ElitePool})$ 
15:       $\mathbf{X}_{new} \leftarrow \text{TLBO\_TEACH}(\mathbf{X}_i, \mathbf{X}_{teacher}, T_F)$ 
16:       $\mathbf{X}_{new} \leftarrow \text{TLBO\_LEARN}(\mathbf{X}_{new}, P)$ 
17:    end if
18:     $\mathbf{X}_{new} \leftarrow \text{CONSTRAINTREPAIR}(\mathbf{X}_{new})$ 
19:     $f_{new} \leftarrow \text{EVALUATE}(\mathbf{X}_{new}) + \text{PENALTY}(\mathbf{X}_{new})$ 
20:    if  $f_{new} < f(\mathbf{X}_i)$  then
21:       $\mathbf{X}_i \leftarrow \mathbf{X}_{new}$ ; 更新  $\text{ElitePool}$ 
22:    end if
23:  end for
24: end for
25: return  $\mathbf{X}_{best}$ 

```

V. 实验与结果分析

A. 实验设置

1) MEC 系统参数配置: 基于实际 MEC 部署场景和 AWS Wavelength、Azure Edge Zones 的配置, 设置参数如表 I 所示。

表 I
MEC 系统参数配置

参数	值
移动设备/边缘/云服务器数量	20/4/2
设备/边缘/云 CPU 频率	0.2-3/4-6/8-12 GHz
传输功率范围	0.1-0.5 W
带宽 (设备-边缘/边缘-云)	10-50/100-1000 Mbps

表 II
任务类型分布及其特征

任务类型	数量 (%)	平均负载 (G)	更新间隔 (s)	最大 AoI(s)
计算密集型	30	1.36	0.949	2.549
数据密集型	25	3.23	1.214	4.114
实时敏感型	25	0.14	0.617	0.690
轻量级	20	0.01	1.302	1.619

2) 算法参数设置: 通过正交实验和网格搜索确定了最优参数配置。公共参数: 种群大小 50, 最大迭代 150 次, 独立运行 5 次, 权重设置 $w^E = 0.25$ 、 $w^T = 0.35$ 和 $w^{AoI} = 0.40$ 。TLBO-HHO 特有参数包括: 初始 HHO 概率 $p_{max} = 0.7$ 、最终 HHO 概率 $p_{min} = 0.1$ 、精英池比例 10%。

B. 算法收敛性分析

图 2 展示了四种算法的收敛曲线对比。TLBO-HHO 在初始阶段 (0-25 次迭代) 快速下降, 中期保持稳定改进, 最终收敛到 0.00102。相比之下, TLBO 收敛稳定但速度慢 (0.00292), GA 易陷入局部最优 (0.00757), GWO 表现不理想 (0.00367)。

C. 性能对比分析

图 3 展示了四种算法的性能指标对比。TLBO-HHO 实现了 0.00102 的平均适应度, 相比 TLBO、GA、GWO 分别提升了 65.2%、85.4%、74.6%。在能耗方面, TLBO-HHO 仅为 3.19J, 显著低于 GA(31.14J)。虽然 GA 在延迟上最优 (0.577s), 但能耗过高; GWO 在 AoI 违规次数最少 (4.6 次), 但综合性能不及 TLBO-HHO。

D. 任务分配策略分析

图 4 展示了任务分配位置对比。TLBO-HHO 采用高度本地化策略 (97.5% 本地执行), 有效降低了传输延迟和能耗。相比之下, GA 过度使用边缘资源 (22.5%), GWO 云端利用率最高 (12.5%)。

E. AoI 性能分析

图 5 展示了不同任务类型的 AoI 性能。TLBO-HHO 对实时敏感型任务控制效果最好, 95% 的 AoI 在 1 秒以内。通过优先本地执行和合理资源分配, 各类任务的 AoI 都保持在可接受范围。

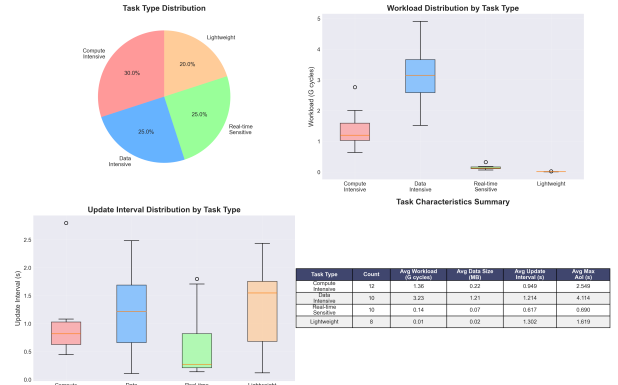


图 5. 任务类型 AoI 性能分析

VI. 结论与未来工作

A. 研究总结

本文针对移动边缘计算中的任务卸载与资源分配优化问题, 提出了一种基于双阶段集成的 TLBO-HHO 融合算法, 并首次将 AoI 纳入 MEC 任务卸载优化目

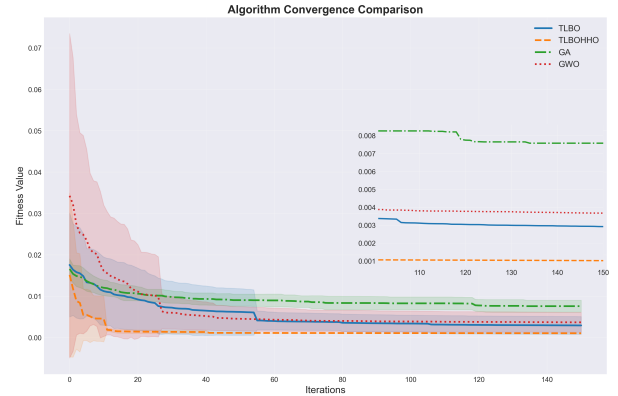


图 2. 算法收敛曲线对比

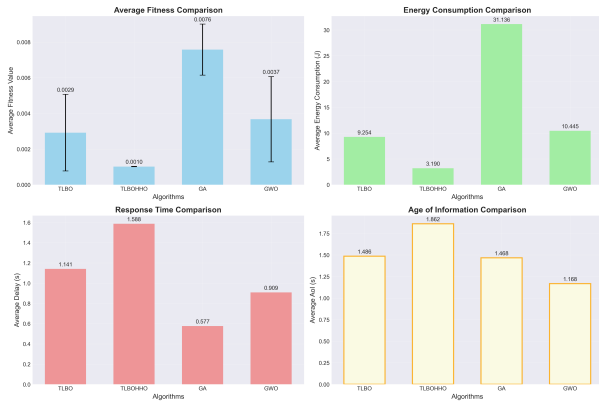


图 3. 算法性能指标对比

标。通过理论分析、算法设计和实验验证，得出以下结论：

问题建模贡献：建立了综合考虑能耗、延迟和 AoI 的三目标优化模型，采用 0.25-0.35-0.40 的权重设置适应信息时效性需求。通过排队论分析了任务等待时间，并通过理论证明确立了问题的 NP-hard 特性，为采用元启发式算法提供了理论依据。模型考虑了实际 MEC 系统的多维约束，包括资源容量、带宽限制、延迟要求、能量预算和 AoI 约束，更贴近实际应用场景。

算法设计创新：提出的双阶段集成 TLBO-HHO 算法通过创新的融合机制实现了性能突破。算法利用 TLBO 的教学阶段引导全局探索，HHO 的狩猎策略提供局部开发，通过自适应概率切换 (0.7→0.1) 和参数协调策略实现了探索与开发的动态平衡。特别是引入的 AoI 感知适应度评估和约束处理机制，使算法能够有效优化信息新鲜度。

性能提升显著：在包含 20 个设备、4 个边缘服务器、2 个云服务器和 40 个任务的实验场景中，TLBO-HHO

相比 TLBO、GA、GWO 分别实现了 65.2%、85.4% 和 74.6% 的适应度值改进。算法在能耗方面表现尤为突出，平均能耗仅为 3.19J，相比 GA 降低了 89.8%。同时，通过 97.5% 的本地执行策略，有效平衡了延迟和 AoI 性能。

实际应用价值：算法展现出智能的任务分配策略，能够根据任务类型自适应调整执行位置。对于实时敏感型任务，95% 的 AoI 控制在 1 秒以内；对于计算密集型任务，通过合理资源分配保证了性能。这种差异化的处理策略符合实际 MEC 应用的需求，具有重要的实用价值。

B. 研究局限性

尽管取得了显著成果，本研究仍存在以下局限：

静态场景假设：当前模型假设任务特征和网络状态相对稳定，未充分考虑用户移动性、信道时变性和任务动态到达。实际 MEC 系统中，这些动态因素对性能有重要影响。

集中式决策架构：算法采用集中式优化，需要全局信息收集和计算。在大规模分布式 MEC 场景下，可能面临通信开销大、单点故障和可扩展性受限等问题。

安全性考虑不足：未充分考虑任务卸载过程中的数据隐私保护、通信安全和恶意攻击防御等安全问题。

AoI 模型简化：当前 AoI 模型假设任务更新间隔固定，未考虑随机到达和非周期性更新的情况。

C. 未来研究方向

基于本文的研究成果和存在的局限性，未来的研究方向包括：

分布式算法实现：研究 TLBO-HHO 的分布式版本，降低通信开销，设计基于共识的分布式优化框架，实现边缘节点间的协作优化。

动态场景扩展：考虑用户移动性和任务动态到达，设计在线优化机制，研究多时间尺度的 AoI 优化策略。

联邦学习集成：结合联邦学习保护用户隐私，设计隐私保护的分布式任务卸载机制，在保证数据安全的同时优化 AoI。

多目标优化深化：研究更复杂的多目标优化框架，考虑服务质量、公平性等其他目标，设计 Pareto 最优解集的生成和选择机制。

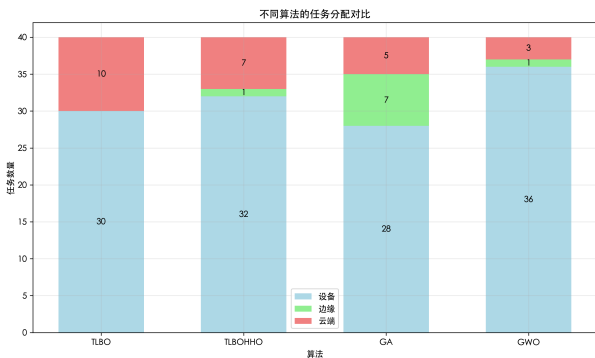


图 4. 算法任务分配对比

实际系统部署：在真实 MEC 测试床上验证算法性能，考虑实际网络协议和系统开销，开发面向产业应用的优化工具。

参考文献

- [1] Mao Y, You C, Zhang J, et al. A survey on mobile edge computing: The communication perspective[J]. IEEE communications surveys & tutorials, 2017, 19(4): 2322-2358.
- [2] Sardellitti S, Scutari G, Barbarossa S. Joint optimization of radio and computational resources for multicell mobile-edge computing[J]. IEEE Transactions on Signal and Information Processing over Networks, 2015, 1(2): 89-103.
- [3] Huang L, Bi S, Zhang Y J A. Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks[J]. IEEE Transactions on Mobile Computing, 2019, 19(11): 2581-2593.
- [4] Guo S, Xiao B, Yang Y, et al. Energy-efficient dynamic offloading and resource scheduling in mobile cloud computing[C]//IEEE INFOCOM 2016. IEEE, 2016: 1-9.
- [5] Rao R V, Savsani V J, Vakharia D P. Teaching-learning-based optimization: a novel method for constrained mechanical design optimization problems[J]. Computer-aided design, 2011, 43(3): 303-315.
- [6] Heidari A A, Mirjalili S, Faris H, et al. Harris hawks optimization: Algorithm and applications[J]. Future generation computer systems, 2019, 97: 849-872.
- [7] Chen H, Heidari A A, Chen H, et al. Multi-population differential evolution-assisted Harris hawks optimization: Framework and case studies[J]. Future Generation Computer Systems, 2020, 111: 175-198.
- [8] Tu B, Wang F, Huo Y, et al. A hybrid algorithm of grey wolf optimizer and harris hawks optimization for solving global optimization problems with improved convergence performance[J]. Scientific Reports, 2023, 13(1): 22909.
- [9] Wang C, Yu F R, Liang C, et al. Joint computation offloading and interference management in wireless cellular networks with mobile edge computing[J]. IEEE Transactions on Vehicular Technology, 2017, 66(8): 7432-7445.
- [10] Liu J, Mao Y, Zhang J, et al. Delay-optimal computation task scheduling for mobile-edge computing systems[C]//IEEE ISIT 2016. IEEE, 2016: 1451-1455.
- [11] Wang L, Wang K, Pan C, et al. Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing[J]. IEEE Transactions on Mobile Computing, 2021, 21(10): 3536-3550.
- [12] Li Z, Zhu Q. Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing[J]. Information, 2020, 11(2): 83.
- [13] Zang L, Zhang X, Guo B. Federated deep reinforcement learning for online task offloading and resource allocation in WPC-MEC networks[J]. IEEE Access, 2022, 10: 9856-9867.
- [14] Ali A, Shah S A A, Al Shloul T, et al. Multiobjective Harris Hawks optimization-based task scheduling in cloud-fog computing[J]. IEEE Internet of Things Journal, 2024, 11(13): 24334-24352.
- [15] Abd-Elmagid M A, Pappas N, Dhillon H S. On the role of age of information in the Internet of Things[J]. IEEE Communications Magazine, 2019, 57(12): 72-77.
- [16] Kaul S, Yates R, Gruteser M. Real-time status: How often should one update?[C]//IEEE INFOCOM 2012. IEEE, 2012: 2731-2735.
- [17] Liu L, Feng J, Mu X, et al. Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing[J]. IEEE Transactions on Intelligent Transportation Systems, 2023, 24(12): 15513-15526.
- [18] Zhou Z, Liu P, Feng J, et al. Computation resource allocation and task assignment optimization in vehicular fog computing: A contract-matching approach[J]. IEEE Transactions on Vehicular Technology, 2019, 68(4): 3113-3125.